Ipule Pipi & Jacob L
CSE: 4020 - Database Systems
Dr. Zahra Nematzadeh
7th May 2026

Documentation of Business Database Creation

Phase 1: Database Creation & Data Loading

The goal of this phase was to prepare and import the dataset into MySQL, enabling efficient structuring and querying.

First, the original Excel dataset was reviewed and cleaned. This involved standardizing date formats to YYYY-MM-DD, removing currency symbols and commas from numeric values, updating column names to a consistent naming scheme, and ensuring all fields were consistent with their intended data types. The cleaned dataset was then saved as a CSV file named CompanyData-cleaned.csv to ensure compatibility with MySQL, so we can extract this data.

Next, a new database named company_db was created using the commands below in Figure 1.1.

```
--creates a new database called company_db
DROP DATABASE IF EXISTS company_db;
CREATE DATABASE company_db;
USE company_db;
```

Figure 1.1: The commands used to create a new database called company_db.

Figure 1.2 below shows that the database has been created successfully.

#	Time	Action	Message	Duration / Fetch
1	13:02:28	DROP DATABASE IF EXISTS company_db	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'company_db'; database doe...	0.062 sec
2	13:02:28	CREATE DATABASE company_db	1 row(s) affected	0.000 sec
3	13:02:28	USE company_db	0 row(s) affected	0.000 sec

Figure 1.2: Output showing the successful creation of company_database.

A table called raw_data was then defined to match the structure of the cleaned dataset. Each column was assigned an appropriate data type (e.g., INT, VARCHAR, DATE, DECIMAL) to ensure proper storage and validation.

The snippet of the commands used is shown in Figure 1.3 (refer to phase1_loading.sql for full command details).

```sql

DROP TABLE IF EXISTS raw_data;
CREATE TABLE raw_data (
    order_id INT,
    customer_id CHAR(5),
    employee_id INT,
    order_date DATE,
    required_date DATE,
    shipped_date DATE,
    ship_via INT,
    freight DECIMAL(10,2),

    product_id INT,
    order_unit_price DECIMAL(10,2),
    quantity INT,
    discount DECIMAL(5,2),

    company_name VARCHAR(255),
    contact_name VARCHAR(255),
    contact_title VARCHAR(255),

    employee_last_name VARCHAR(100),
    employee_first_name VARCHAR(100),
    employee_title VARCHAR(100),

    product_name VARCHAR(255),
    supplier_id INT,
    category_id INT,
    quantity_per_unit VARCHAR(100),
    catalog_unit_price DECIMAL(10,2),
    units_in_stock INT,
    units_on_order INT,
    reorder_level INT,
```

Figure 1.3: Commands used to create the raw_data table.

Figure 1.4 below confirms the creation of the raw_data table, which is currently empty. We ensured the schema matched the exact attribute structure in the Excel sheet so MySQL could correctly map and extract them.

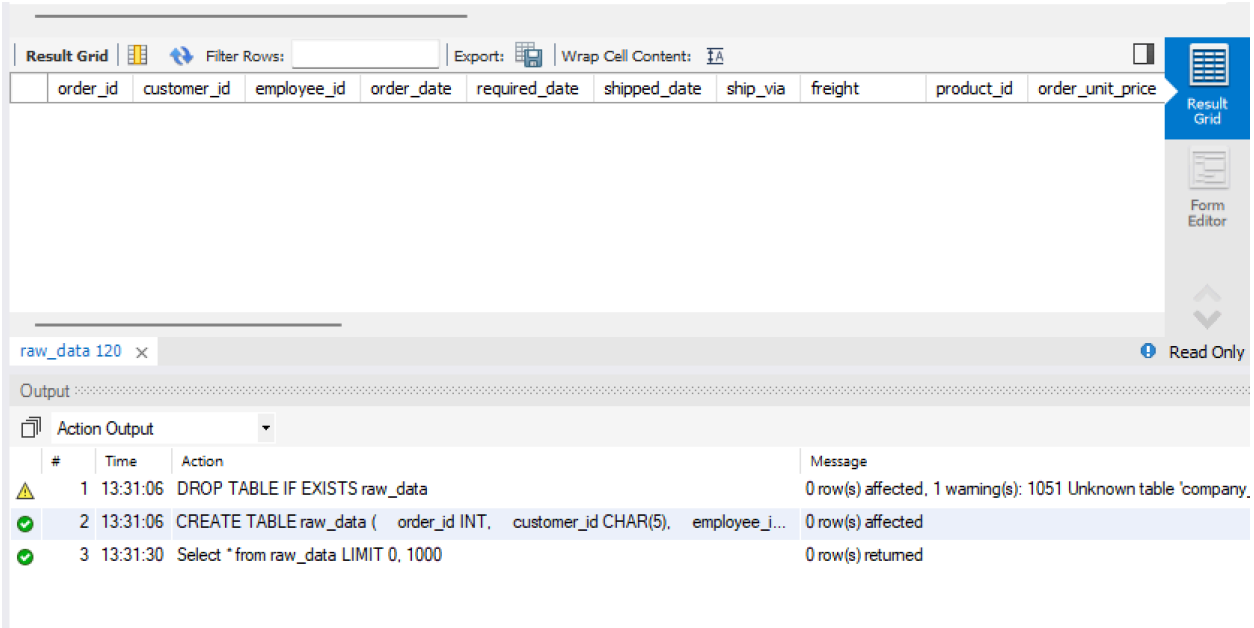


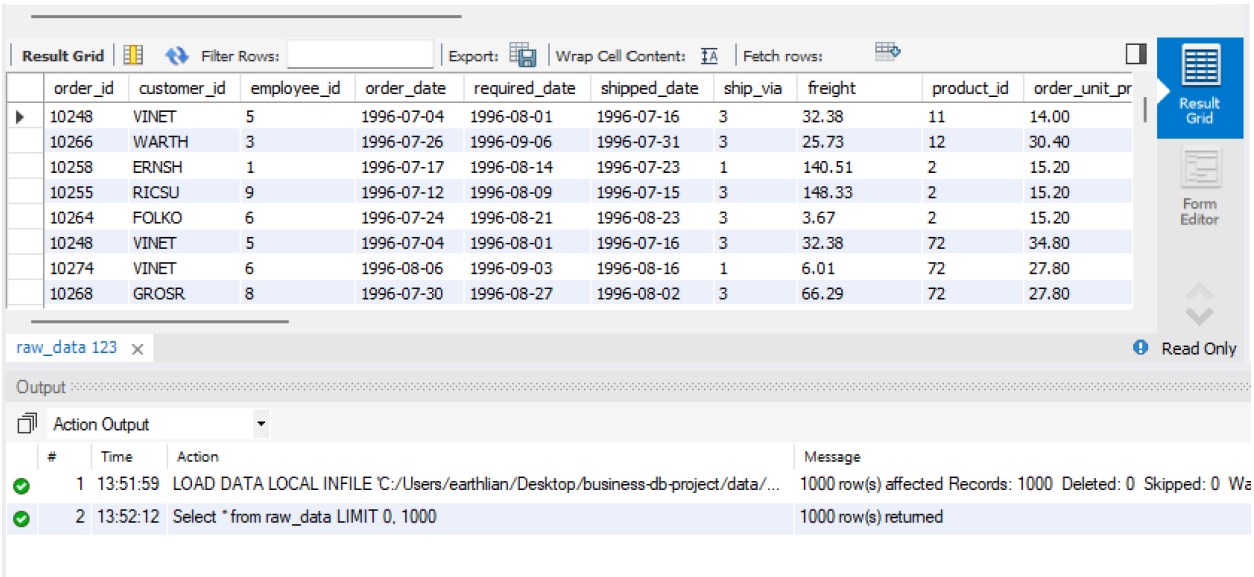
Figure 1.4: Out showing the creation of the raw_data table.

After defining the table structure, the cleaned CSV file was imported into the MySQL raw_data table using the LOAD DATA LOCAL INFILE command, as shown in Figure 1.5. This allowed efficient loading of all rows from the CSV file into the raw_data table.

```
LOAD DATA LOCAL INFILE 'C:/Users/earthlian/Desktop/business-db-project/data/cleaned/CompanyData_cleaned.csv'
INTO TABLE raw_data
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Figure 1.5: Command used to load the CSV file into the raw_data table

Figure 1.6 below shows the output confirming that the loading process was successful, along with the table results. The full table result can be found in the phase1_raw-table.csv file.



order_id	customer_id	employee_id	order_date	required_date	shipped_date	ship_via	freight	product_id	order_unit_pr
10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38	11	14.00
10266	WARTH	3	1996-07-26	1996-09-06	1996-07-31	3	25.73	12	30.40
10258	ERNSH	1	1996-07-17	1996-08-14	1996-07-23	1	140.51	2	15.20
10255	RICSU	9	1996-07-12	1996-08-09	1996-07-15	3	148.33	2	15.20
10264	FOLKO	6	1996-07-24	1996-08-21	1996-08-23	3	3.67	2	15.20
10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38	72	34.80
10274	VINET	6	1996-08-06	1996-09-03	1996-08-16	1	6.01	72	27.80
10268	GROSR	8	1996-07-30	1996-08-27	1996-08-02	3	66.29	72	27.80

Figure 1.6: Output and table showing the successful extraction.

Finally, we ran the command below to display the total count and verify that the data import was successful:

SELECT COUNT(*) FROM raw_data;

This confirmed that all 1000 rows were successfully loaded, as shown in Figure 1.7 below.

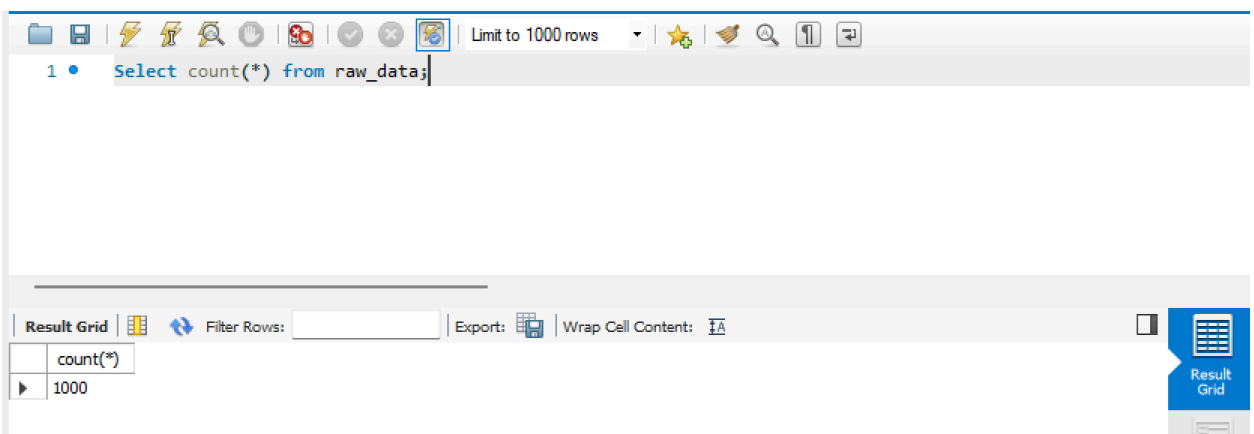


Figure 1.7: Output showing a count of 1000, showing successful extraction.

This phase established a clean, structured, and intuitive foundation for future database design and normalization.

Phase 2: Database Diagram Design (ER Diagram)

The goal of this phase was to design an Entity-Relationship (ER) Diagram to model the business database structure and visualize the relationships among entities.

After examining the imported dataset, the main entities involved in the business operations were identified. These entities included: Customers, Employees, Orders, Order Details, Products, Suppliers, and Categories

Each entity was analyzed to determine its attributes and relationships with other entities. Primary keys (PK) uniquely identify records, while foreign keys (FK) establish relationships between tables.

The ER diagram was designed to represent the real-world business workflow. For example: A customer can place many orders, an employee can handle many orders, an order can contain many order items, a product can appear in many order items, a supplier can supply many products, and a category can classify many products.

The OrderDetails table was especially important because it resolved the many-to-many relationship between orders and products. Instead of storing multiple products inside a single order record, each product associated with an order was stored as a separate row in the orderDetails table. This improved scalability, flexibility, and normalization.

The ER diagram also included the major attributes of each entity, along with the appropriate primary key and foreign key relationships.

A rough draft of the ER diagram was generated in draw.io and is shown in Figure 2.1 below.

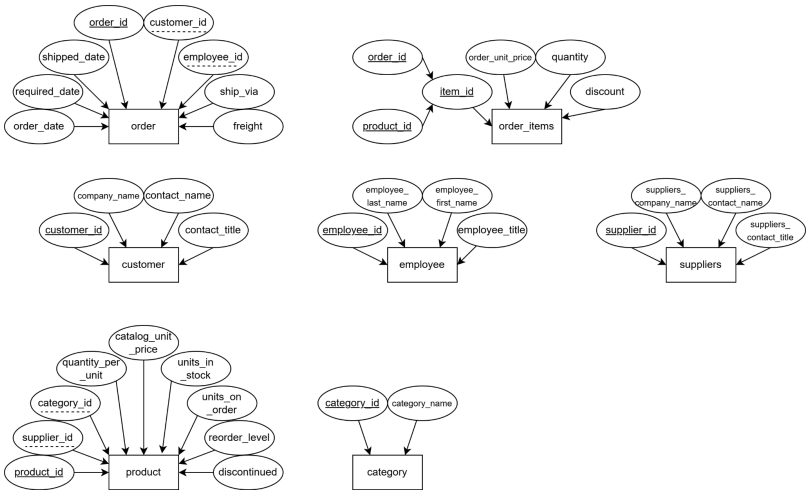


Figure 2.1: ER diagram rough draft.

The finished diagram was generated using Mermaid ER notation and exported as an image for documentation purposes using the command shown in the snippet shown in Figure 2.2 (er_diagram.mmd).

```
erDiagram
    CUSTOMERS ||--o{ ORDERS : places
    EMPLOYEES ||--o{ ORDERS : handles
    ORDERS ||--o{ ORDER_DETAILS : contains
    PRODUCTS ||--o{ ORDER_DETAILS : included_in
    SUPPLIERS ||--o{ PRODUCTS : supplies
    CATEGORIES ||--o{ PRODUCTS : classifies

    CUSTOMERS {
        int customer_id PK
        varchar company_name
        varchar contact_name
        varchar contact_title
    }

    EMPLOYEES {
        int employee_id PK
        varchar employee_first_name
        varchar employee_last_name
        varchar employee_title
    }

    SUPPLIERS {
        int supplier_id PK
        varchar suppliers_company_name
        varchar suppliers_contact_name
        varchar suppliers_contact_title
    }

    CATEGORIES {
        int category_id PK
        varchar category_name
    }
```

The finalized ER diagram shown in Figure 2.2 provided a clear blueprint for the normalization and table alteration phases that followed.

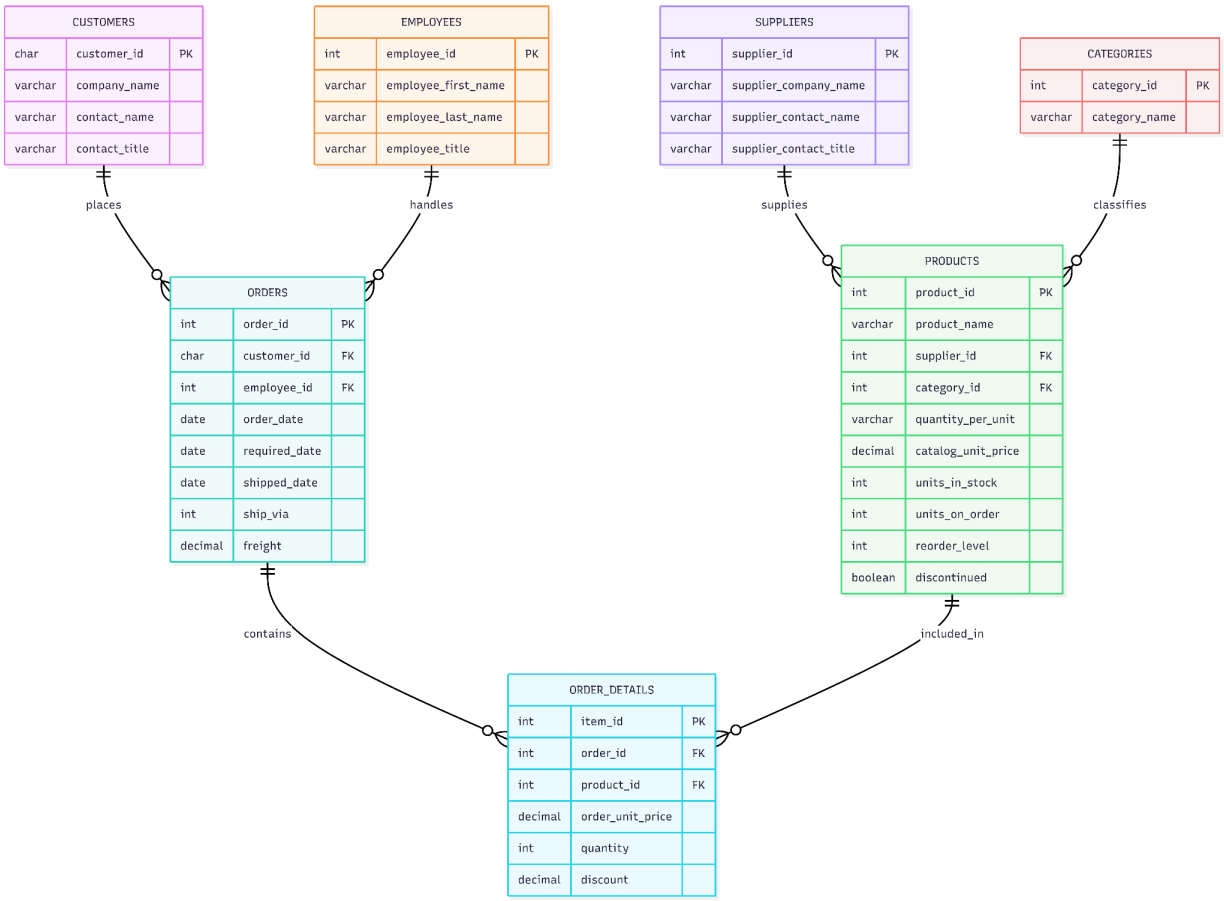


Figure 2.2: The finalized ER diagram.

Phase 3: Data Normalization

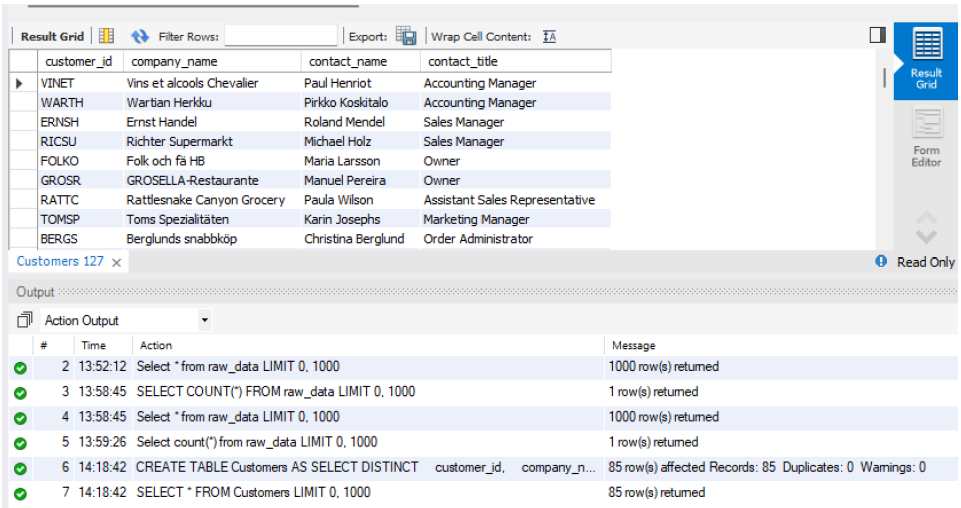
First, we extracted customer data into a separate Customers table by using this command shown in the snippet below in Figure 3.1.

```
CREATE TABLE Customers AS
SELECT DISTINCT
    customer_id,
    company_name,
    contact_name,
    contact_title
FROM raw_data;

SELECT * FROM Customers;
```

Figure 3.1: Commands to extract customer data from raw_data and store it in the Customers table.

Figure 3.2 confirms that the extraction of customer data was successful (see Customers-table.csv for the entire customer table).



customer_id	company_name	contact_name	contact_title
VINET	Vins et alcools Chevalier	Paul Henriot	Accounting Manager
WARTH	Wartian Herkku	Pirkko Koskitalo	Accounting Manager
ERNSH	Ernst Handel	Roland Mendel	Sales Manager
RICSU	Richter Supermarkt	Michael Holz	Sales Manager
FOLKO	Folk och fä HB	Maria Larsson	Owner
GROSR	GROSELLA-Restaurante	Manuel Pereira	Owner
RATTC	Rattlesnake Canyon Grocery	Paula Wilson	Assistant Sales Representative
TOMSP	Toms Spezialitäten	Karin Josephs	Marketing Manager
BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator

#	Time	Action	Message
2	13:52:12	Select * from raw_data LIMIT 0, 1000	1000 row(s) returned
3	13:58:45	SELECT COUNT(*) FROM raw_data LIMIT 0, 1000	1 row(s) returned
4	13:58:45	Select * from raw_data LIMIT 0, 1000	1000 row(s) returned
5	13:59:26	Select count(*) from raw_data LIMIT 0, 1000	1 row(s) returned
6	14:18:42	CREATE TABLE Customers AS SELECT DISTINCT customer_id, company_n...	85 row(s) affected Records: 85 Duplicates: 0 Warnings: 0
7	14:18:42	SELECT * FROM Customers LIMIT 0, 1000	85 row(s) returned

Figure 3.2: Customer table.

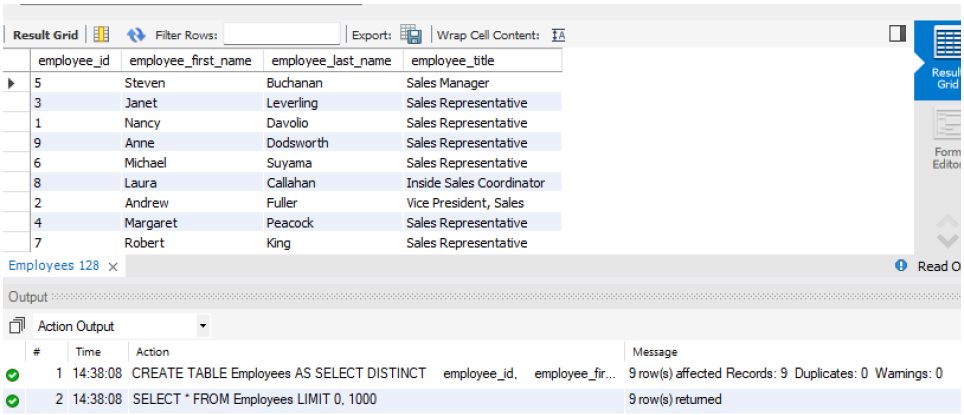
Next, data from employees was extracted and stored in the Employees table using the command below in Figure 3.3.

```
CREATE TABLE Employees AS
SELECT DISTINCT
    employee_id,
    employee_first_name,
    employee_last_name,
    employee_title
FROM raw_data;

SELECT * FROM Employees;
```

Figure 3.3: Commands used to extract employee data and display it.

Figure 3.4 shows the successful extraction of employee data into the Employees table (refer to Employees-table.csv to view the entire Employees table).



employee_id	employee_first_name	employee_last_name	employee_title
5	Steven	Buchanan	Sales Manager
3	Janet	Leverling	Sales Representative
1	Nancy	Davolio	Sales Representative
9	Anne	Dodsworth	Sales Representative
6	Michael	Suyama	Sales Representative
8	Laura	Callahan	Inside Sales Coordinator
2	Andrew	Fuller	Vice President, Sales
4	Margaret	Peacock	Sales Representative
7	Robert	King	Sales Representative

#	Time	Action	Message
1	14:38:08	CREATE TABLE Employees AS SELECT DISTINCT employee_id, employee_fir...	9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0
2	14:38:08	SELECT * FROM Employees LIMIT 0, 1000	9 row(s) returned

Figure 3.4: Employees table

Again, we did the same for Orders.

```
CREATE TABLE Orders AS
SELECT DISTINCT
    order_id,
    customer_id,
    employee_id,
    order_date,
    required_date,
    shipped_date,
    ship_via,
    freight
FROM raw_data;

SELECT * FROM Orders;
```

Figure 3.5: Commands for extracting orders data

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	employee_id	order_date	required_date	shipped_date	ship_via	freight
10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38
10266	WARTH	3	1996-07-26	1996-09-06	1996-07-31	3	25.73
10258	ERNSH	1	1996-07-17	1996-08-14	1996-07-23	1	140.51
10255	RICSU	9	1996-07-12	1996-08-09	1996-07-15	3	148.33
10264	FOLKO	6	1996-07-24	1996-08-21	1996-08-23	3	3.67
10274	VINET	6	1996-08-06	1996-09-03	1996-08-16	1	6.01
10268	GROSR	8	1996-07-30	1996-08-27	1996-08-02	3	66.29
10272	RATTC	6	1996-08-02	1996-08-30	1996-08-06	2	98.03
10262	RATTC	8	1996-07-22	1996-08-19	1996-07-25	3	48.29

Orders 129 | Read Only

Output

Action Output

#	Time	Action	Message
1	14:47:00	CREATE TABLE Orders AS SELECT DISTINCT order_id, customer_id, emplo...	378 row(s) affected Records: 378 Duplicates: 0 Warnings: 0
2	14:47:00	SELECT * FROM Orders LIMIT 0, 1000	378 row(s) returned

Figure 3.6: Orders table.

As well as for Order details data.

```
CREATE TABLE OrderDetails AS
SELECT DISTINCT
    order_id,
    product_id,
    order_unit_price,
    quantity,
    discount
FROM raw_data;

SELECT * FROM OrderDetails;
```

Figure 3.7: Commands for extracting order details.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

order_id	product_id	order_unit_price	quantity	discount
10248	11	14.00	12	0.00
10266	12	30.40	12	0.05
10258	2	15.20	50	0.20
10255	2	15.20	20	0.00
10264	2	15.20	35	0.00
10248	72	34.80	5	0.00
10274	72	27.80	7	0.00
10268	72	27.80	4	0.00
10272	72	27.80	24	0.00

OrderDetails 130 | Read Only

Output

Action Output

#	Time	Action	Message
1	14:52:58	CREATE TABLE OrderDetails AS SELECT DISTINCT order_id, product_id, or...	1000 row(s) affected Records: 1000 Duplicates: 0 Warnings: 0
2	14:52:58	SELECT * FROM OrderDetails LIMIT 0, 1000	1000 row(s) returned

Figure 3.8: Orders Details table.

Again, the same applied to suppliers and categories data.

```
CREATE TABLE Suppliers AS
SELECT DISTINCT
    supplier_id,
    supplier_company_name,
    supplier_contact_name,
    supplier_contact_title
FROM raw_data;

SELECT * FROM Suppliers;
```

Figure 3.9: Commands for extracting supplier data.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

supplier_id	supplier_company_name	supplier_contact_name	supplier_contact_title
5	Cooperativa de Quesos 'Las Cabras'	Antonio del Valle Saavedra	Export Administrator
1	Exotic Liquids	Charlotte Cooper	Purchasing Manager
14	Formaggi Fortini s.r.l.	Elio Rossi	Sales Representative
2	New Orleans Cajun Delights	Shelley Burke	Order Administrator
3	Grandma Kelly's Homestead	Regina Murphy	Sales Representative
6	Mayumi's	Mayumi Ohno	Marketing Representative
7	Pavlova	Ltd.	Ian Devling
20	Leka Trading	Chandra Leka	Owner
15	Norske Meierier	Beate Vileid	Marketing Manager

Suppliers 131 × Read On

Output

Action Output

#	Time	Action	Message
1	15:00:06	CREATE TABLE Suppliers AS SELECT DISTINCT supplier_id, supplier_compan...	29 row(s) affected Records: 29 Duplicates: 0 Warnings: 0
2	15:00:06	SELECT * FROM Suppliers LIMIT 0, 1000	29 row(s) returned

Figure 3.10: Suppliers Table.

```
CREATE TABLE Categories AS
SELECT DISTINCT
    category_id,
    category_name
FROM raw_data;

SELECT * FROM Categories;
```

Figure 3.11: Command for extracting categories data.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_id	category_name
4	Dairy Products
1	Beverages
2	Condiments
7	Produce
5	Grains/Cereals
8	Seafood
6	Meat/Poultry
3	Confections

Categories 132 × Rea

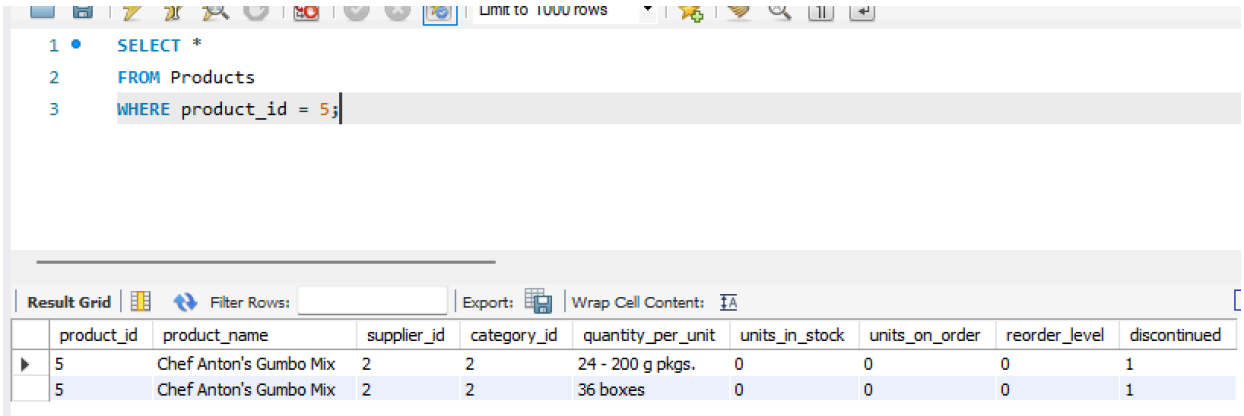
Output

Action Output

#	Time	Action	Message
1	15:03:39	CREATE TABLE Categories AS SELECT DISTINCT category_id, category_nam...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0
2	15:03:39	SELECT * FROM Categories LIMIT 0, 1000	8 row(s) returned

Figure 3.12: Categories table.

When adding primary key constraints to the products table, duplicate product records were found for the same product_id because quantity_per_unit had inconsistent values for product_id 5, as shown in Figure 3.13.



product_id	product_name	supplier_id	category_id	quantity_per_unit	units_in_stock	units_on_order	reorder_level	discontinued
5	Chef Anton's Gumbo Mix	2	2	24 - 200 g pkgs.	0	0	0	1
5	Chef Anton's Gumbo Mix	2	2	36 boxes	0	0	0	1

Figure 3.13: Duplicate error when adding primary key constraint to product.

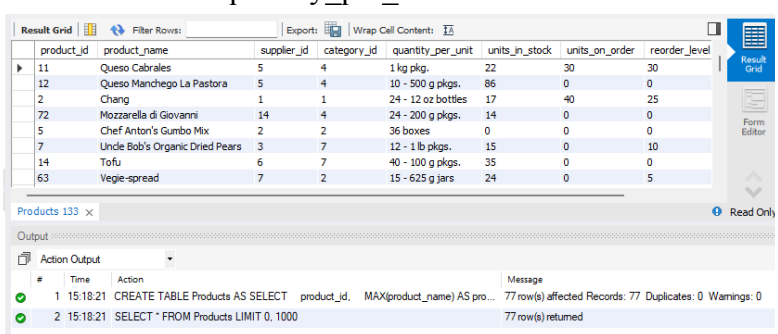
Since product_id represents a product's identity, the Products table was rebuilt using GROUP BY product_id to enforce one row per product, and aggregate functions, such as MAX(), were used to select one representative value for each product attribute, as shown in Figure 3.14, to fix the issue.

```
CREATE TABLE Products AS
SELECT
    product_id,
    MAX(product_name) AS product_name,
    MAX(supplier_id) AS supplier_id,
    MAX(category_id) AS category_id,
    MAX(quantity_per_unit) AS quantity_per_unit,
    MAX(units_in_stock) AS units_in_stock,
    MAX(units_on_order) AS units_on_order,
    MAX(reorder_level) AS reorder_level,
    MAX(discontinued) AS discontinued
FROM raw_data
GROUP BY product_id;

SELECT * FROM Products;
```

Figure 3.14: Updated command used to extract product data and handle duplicates.

Please note that this is a temporary fix; in a real-world scenario, the company must be consulted to confirm the correct value for the quantity_per_unit attribute.



product_id	product_name	supplier_id	category_id	quantity_per_unit	units_in_stock	units_on_order	reorder_level
11	Queso Cabrales	5	4	1 kg pkg.	22	30	30
12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	86	0	0
2	Chang	1	1	24 - 12 oz bottles	17	40	25
72	Mozzarella di Giovanni	14	4	24 - 200 g pkgs.	14	0	0
5	Chef Anton's Gumbo Mix	2	2	36 boxes	0	0	0
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	15	0	10
14	Tofu	6	7	40 - 100 g pkgs.	35	0	0
63	Vegie-spread	7	2	15 - 625 g jars	24	0	5

Figure 3.15: Product table with fixed duplicates for product with id 5.

Phase 4: Table Alteration

First, the original table data types and constraints were identified before applying additional constraints and data types using the command shown in Figure 4.1.

```
USE company_db;

DESCRIBE Customers;
```

Figure 4.1: Command used to describe customers before adding constraints.

Figure 4.2 shows the output of the commands and describes the current customers table.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	char(5)	YES		NULL	
company_name	varchar(255)	YES		NULL	
contact_name	varchar(255)	YES		NULL	
contact_title	varchar(255)	YES		NULL	

Result 135

Output

Action Output

#	Time	Action	Message
1	15:10:42	USE company_db	0 row(s) affected
2	15:10:42	DESCRIBE Customers	4 row(s) returned

Figure 4.2: Output of the customers table description.

As shown in Figure 4.2, no constraints have been added yet, and only the data types are defined, as they were when the extraction was implemented in phase 1.

Next, duplicates of the customer id were checked before implementing the primary key constraint using the command shown in Figure 4.3.

```
SELECT customer_id, COUNT(*) AS count
FROM Customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Figure 4.3: Commands for checking duplicates in customer_id

Figure 4.4 below shows that there are no duplicates.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	count

Figure 4.4. Output showing no customer_id duplicates.

The null check was performed to ensure no customer was missing an ID before setting customer_id to not null using the command shown in Figure 4.5.

```
SELECT *
FROM Customers
WHERE customer_id IS NULL;
```

Figure 4.5: Command for null check.

Figure 4.6 shows that no customer_id is null.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	company_name	contact_name	contact_title

Figure 4.6: Output showing now rows returned for null customer_id.

The same was applied to the company name, yielding a similar result; the primary key and not null constraints were then applied to the customer table using the command shown in Figure 4.7.

```
--apply constraints
ALTER TABLE Customers
MODIFY customer_id CHAR(5) NOT NULL,
MODIFY company_name VARCHAR(255) NOT NULL,
ADD PRIMARY KEY (customer_id);
```

Figure 4.7: Command used to enforce a constraint in the customer table.

Figure 4.8 below confirms that the constraints have been applied successfully.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	char(5)	NO	PRI	NULL	
company_name	varchar(255)	NO		NULL	
contact_name	varchar(255)	YES		NULL	
contact_title	varchar(255)	YES		NULL	

Figure 4.8: Output showing constraints applied to the customer table.

The same validation and constraint implementation process was then applied to the remaining tables: Employees, Suppliers, Categories, Products, Orders, and OrderDetails. The complete SQL commands for all table alterations are provided in phase4-table-alterations.sql.

The exported output of the Customers table after constraint implementation was stored in the file phase4_customer-table.csv. Exported outputs for the remaining altered tables were also stored in their respective files, including phase4_employee-table.csv, phase4_supplier-table.csv, and other related Phase 4 output files.

The Employees table was altered to enforce entity integrity and required employee information. The employee_id attribute was defined as the primary key and set to NOT NULL. Additionally, employee_first_name and employee_last_name were enforced as non-null attributes to ensure that each employee record contains identifying information. These changes are shown in Figure 4.9 below.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
employee_id	int	NO	PRI	NULL	
employee_first_name	varchar(100)	NO		NULL	
employee_last_name	varchar(100)	NO		NULL	
employee_title	varchar(100)	YES		NULL	

Figure 4.9: Output reflecting the changes made to Employees.

The Suppliers table was altered to enforce entity integrity and required supplier information. The supplier_id attribute was defined as the primary key and set to NOT NULL, as shown in Figure 4.10 below. Additionally, supplier_company_name was required to ensure that every supplier record contains identifying company information.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
supplier_id	int	NO	PRI	NULL	
supplier_company_name	varchar(255)	NO		NULL	
supplier_contact_name	varchar(255)	YES		NULL	
supplier_contact_title	varchar(255)	YES		NULL	

Figure 4.10: Output reflecting the changes made to Suppliers.

The Categories table was altered to enforce entity integrity and domain consistency. The category_id attribute was defined as the primary key and set to NOT NULL. The category_name attribute was also enforced as NOT NULL and assigned a UNIQUE constraint to prevent duplicate category names. These changes are shown in Figure 4.11 below.

Result Grid | Filter Rows: | Export: | Wrap Cel

Field	Type	Null	Key	Default	Extra
category_id	int	NO	PRI	NULL	
category_name	varchar(100)	NO	UNI	NULL	

Figure 4.11: Output reflecting the changes made to Categories.

The Products table was altered to enforce entity integrity and referential integrity. The product_id attribute was defined as the primary key and set to NOT NULL. The product_name, supplier_id, and category_id attributes were also made non-null. Foreign key constraints were added so that each product must reference an existing supplier and category. These changes are shown in Figure 4.12 below.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	
product_name	varchar(255)	NO		NULL	
supplier_id	int	NO	MUL	NULL	
category_id	int	NO	MUL	NULL	
quantity_per_unit	varchar(100)	YES		NULL	
units_in_stock	int	YES		NULL	
units_on_order	int	YES		NULL	
reorder_level	int	YES		NULL	
discontinued	tinyint(1)	YES		NULL	

Figure 4.12: Output reflecting the changes made to Products.

The Orders table was altered to enforce entity integrity and referential integrity. The order_id attribute was defined as the primary key and set to NOT NULL. The customer_id, employee_id, and order_date attributes were also enforced as non-null. Foreign key constraints were added to ensure that each order references valid customer and employee records. These changes are shown in Figure 4.13 below.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_id	char(5)	NO	MUL	NULL	
employee_id	int	NO	MUL	NULL	
order_date	date	NO		NULL	
required_date	date	YES		NULL	
shipped_date	date	YES		NULL	
ship_via	int	YES		NULL	
freight	decimal(10,2)	YES		NULL	

Figure 4.13: Output reflecting the changes made to Orders.

The OrderDetails table was altered to enforce entity integrity and referential integrity. The order_id and product_id attributes were combined to form a composite primary key, ensuring that each product appears only once per order. Foreign key constraints were added to enforce valid relationships with the Orders and Products tables. Additionally, pricing and quantity-related attributes were enforced as non-null to maintain complete transaction records. These changes are shown in Figure 4.14 below.

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
product_id	int	NO	PRI	NULL	
order_unit_price	decimal(10,2)	NO		NULL	
quantity	int	NO		NULL	
discount	decimal(5,2)	NO		NULL	

Figure 4.14: Output reflecting the changes made to OrderDetails.

Phase 5: Views, Triggers, and Stored Procedures

5.1: View

Before creating the view, the commands shown in Figure 5.1.1 below were executed to verify that the correct result is shown. This check ensures that the data is correct before creating a view that would show incorrect data.

```sql
-- Preview the query result before creating the view
SELECT
    e.employee_id,
    e.employee_first_name,
    e.employee_last_name,
    SUM(od.quantity) AS total_quantity_sold,
    SUM(od.order_unit_price * od.quantity) AS total_revenue
FROM Employees e
JOIN Orders o
    ON e.employee_id = o.employee_id
JOIN OrderDetails od
    ON o.order_id = od.order_id
GROUP BY
    e.employee_id,
    e.employee_first_name,
    e.employee_last_name;
```

Figure 5.1.1: Set of commands to verify query result before view creation.

Upon execution, the changes were reflected in the Employees table, shown in Figure 5.1.2 below.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	employee_id	employee_first_name	employee_last_name	total_quantity_sold	total_revenue
▶	1	Nancy	Davolio	3805	95151.38
	2	Andrew	Fuller	2784	66519.00
	3	Janet	Leverling	3421	86548.05
	4	Margaret	Peacock	5301	132910.29
	5	Steven	Buchanan	1549	39327.40
	6	Michael	Suyama	1745	37812.64
	7	Robert	King	1881	59541.60
	8	Laura	Callahan	2639	56056.00
	9	Anne	Dodsworth	1011	18885.30

Figure 5.1.2: Shows actual sales generated, not in the view.

A view named employee_sales_summary was created to display the total quantity sold and total revenue generated by each employee. The view combines the Employees, Orders, and OrderDetails tables using JOIN operations and aggregate functions, such as SUM(). This allows the business owner to easily analyze employee sales performance via a reusable virtual table.

```sql
-- Create a view that shows total quantity sold and total rev
CREATE VIEW employee_sales_summary AS
SELECT
    e.employee_id,
    e.employee_first_name,
    e.employee_last_name,
    SUM(od.quantity) AS total_quantity_sold,
    SUM(od.order_unit_price * od.quantity) AS total_revenue
FROM Employees e
JOIN Orders o
    ON e.employee_id = o.employee_id
JOIN OrderDetails od
    ON o.order_id = od.order_id
GROUP BY
    e.employee_id,
    e.employee_first_name,
    e.employee_last_name;

-- Query the view to verify the output
SELECT *
FROM employee_sales_summary;
```

Figure 5.1.3: Set of commands to create the employee_sales_summary view.

Upon execution, the view looks as shown in Figure 5.1.4 below.

employee_id	employee_first_name	employee_last_name	total_quantity_sold	total_revenue
1	Nancy	Davolio	3805	95151.38
2	Andrew	Fuller	2784	66519.00
3	Janet	Leverling	3421	86548.05
4	Margaret	Peacock	5301	132910.29
5	Steven	Buchanan	1549	39327.40
6	Michael	Suyama	1745	37812.64
7	Robert	King	1881	59541.60
8	Laura	Callahan	2639	56056.00
9	Anne	Dodsworth	1011	18885.30

Figure 5.1.4: Output of the employee_sales_summary view.

5.2: Trigger

Before creating the trigger, the product with product_id 1 was checked to retrieve its details, especially the unit_in_stock value, so it could be used to test whether the trigger executed successfully. The commands used to create it are shown in Figure 5.2.1, and the output of this check is shown in Figure 5.2.2 below.

```
SELECT product_id, product_name, units_in_stock
FROM Products
WHERE product_id = 1;
```

Figure 5.2.1: Checking the details of a product stock before applying the trigger

Result Grid | Filter Rows:

product_id	product_name	units_in_stock
1	Chai	39
NULL	NULL	NULL

Figure 5.2.2: Showing product with id 1 has stock 39.

A trigger named trg_reduce_stock_after_orderdetail_insert was created to automatically reduce product stock quantities whenever a new order detail record is inserted. The trigger activates AFTER INSERT on the OrderDetails table and updates the corresponding product's units_in_stock value in the Products table. Testing confirmed that stock quantities were automatically reduced when a new order detail record was inserted. The commands used to create it are shown in Figure 5.2.3 below.

```
CREATE TRIGGER trg_reduce_stock_after_orderdetail_insert
AFTER INSERT ON OrderDetails
FOR EACH ROW
BEGIN
    UPDATE Products
    SET units_in_stock = units_in_stock - NEW.quantity
    WHERE product_id = NEW.product_id;
END //

DELIMITER ;

-- Verify the trigger was created successfully
SHOW TRIGGERS;
```

Figure 5.2.3: Set of commands used to create the trg_reduce_stock_after_orderdetail_insert trigger.

Figure 5.2.4 below shows details of the trg_reduce_stock_after_orderdetail_insert trigger, including the event, the affected table, the statement itself, when it activates, and when it was created.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Trigger	Event	Table	Statement	Timing	Created
trg_reduce_stock_after_orderdetail_insert	INSERT	orderdetails	BEGIN UPDATE Products SET units_in_stoc...	AFTER	2026-05-08 18:32:5

Figure 5.2.4: Showing trigger details.

To ensure the trigger was working correctly, we created a test that added a new order for product_id 1, then confirmed it. The commands used to create it are shown in Figure 5.2.5, and the test output is shown in Figure 5.2.6 below.

```
-- Test the trigger by inserting a new order detail for product_id = 1
INSERT INTO OrderDetails (
    order_id,
    product_id,
    order_unit_price,
    quantity,
    discount
)
VALUES (
    10248,
    1,
    18.00,
    2,
    0
);

-- Confirm stock was reduced by 2 after the insert
SELECT product_id, product_name, units_in_stock
FROM Products
WHERE product_id = 1;
```

Figure 5.2.5: Set of commands used to create an order with product_id of 1 to test the trigger.

Result Grid | Filter Rows:

product_id	product_name	units_in_stock
1	Chai	37
NULL	NULL	NULL

Figure 5.2.6: Output after inserting an order confirming that unit_in_stock was reduced.

5.3: Stored procedure

Before implementing the stored procedure, a check was performed to find a product with a stock unit count below 10 to confirm that it was implemented correctly. The set of commands used to create it is shown in Figure 5.3.1, and the output of this check is shown in Figure 5.3.2 below.

```
SELECT
    product_id,
    product_name,
    units_in_stock
FROM Products
WHERE units_in_stock < 10;
```

Figure 5.3.1: Set of commands to find products with stock units below 10.

product_id	product_name	units_in_stock
5	Chef Anton's Gumbo Mix	0
17	Alice Mutton	0
29	Thüringer Rostbratwurst	0
31	Gorgonzola Telino	0
53	Perth Pasties	0
21	Sir Rodney's Scones	3
66	Louisiana Hot Spiced Okra	4
74	Longlife Tofu	4
45	Rogede sild	5
8	Northwoods Cranberry Sauce	6
68	Scottish Longbreads	6
32	Mascarpone Fabioli	9
NULL	NULL	NULL

Figure 5.3.2: Output showing products with stock lower than 10.

A stored procedure named GetLowStockProducts was created to identify products requiring restocking. The procedure accepts a stock threshold parameter and returns all products whose units_in_stock value falls below the specified threshold. This allows the business owner to dynamically monitor low inventory levels and support inventory management decisions.

```
-- Create a stored procedure that returns products below a specified stock threshold
DELIMITER //

CREATE PROCEDURE GetLowStockProducts(IN stock_threshold INT)
BEGIN
    SELECT
        product_id,
        product_name,
        units_in_stock
    FROM Products
    WHERE units_in_stock < stock_threshold
    ORDER BY units_in_stock ASC;
END //

DELIMITER ;

-- Call the stored procedure with a threshold of 10 to list products that need restocking
CALL GetLowStockProducts(10);
```

Figure 5.3.3: Command to create stored procedures and calls to that procedure.

Result Grid | Filter Rows: | Export: | Wrap C

product_id	product_name	units_in_stock
5	Chef Anton's Gumbo Mix	0
17	Alice Mutton	0
29	Thüringer Rostbratwurst	0
31	Gorgonzola Telino	0
53	Perth Pasties	0
21	Sir Rodney's Scones	3
66	Louisiana Hot Spiced Okra	4
74	Longlife Tofu	4
45	Rogede sild	5
8	Northwoods Cranberry Sauce	6
68	Scottish Longbreads	6
32	Mascarpone Fabioli	9

Figure 5.3.4: The output confirming the successful execution of the stored procedure.

Phase 6: User Management and Privileges

A DBA user named IpuleDBA was created and granted full administrative privileges on the company_db database using the GRANT ALL PRIVILEGES command. The assigned permissions allow the user to fully manage database objects, modify data, and administer the database environment.

```
CREATE USER 'IpuleDBA'@'localhost'
IDENTIFIED BY 'StrongPassword123!';
```

Figure 6.1: Command to create a database user

	user	host
▶	IpuleDBA	localhost

Figure 6.2: Output showing that database users exist.

```
GRANT ALL PRIVILEGES
ON company_db.*
TO 'IpuleDBA'@'localhost';
```

Figure 6.3: Command to give administrators privileges to the user.

```
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'IpuleDBA'@'localhost';
```

Figure 6.4: Command to apply privilege and show.

	Grants for IpuleDBA@localhost
▶	GRANT USAGE ON *.* TO `IpuleDBA`@`localh...
	GRANT ALL PRIVILEGES ON `company_db`.* T...

Figure 6.5: Showing administrator privilege to the company_db.

An Analyst user named JacobAnalyst was created with read-only access to the company_db database. The user was granted only SELECT privileges, ensuring that data can be queried without allowing modifications to tables or database structures. This follows the principle of least privilege and improves database security.

```
CREATE USER 'JacobAnalyst'@'localhost'
IDENTIFIED BY 'Analyst123!';
```

Figure 6.6: Set of commands for creating an analyst user

Result Grid | Filter Rows:

	user	host
▶	JacobAnalyst	localhost

Figure 6.7: Output verifying the analyst user exists

```
GRANT SELECT
ON company_db.*
TO 'JacobAnalyst'@'localhost';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'JacobAnalyst'@'localhost';
```

Figure 6.8: Command to give the analyst read-only access.

	Grants for JacobAnalyst@localhost
▶	GRANT USAGE ON *.* TO `JacobAnalyst`@`lo...
	GRANT SELECT ON `company_db`.* TO `Jaco...

Figure 6.9: Output showing the analyst user is granted read-only access.

Phase 7: Database Backup

A full database backup was successfully created using MySQL Workbench's Data Export feature. The backup included database structure, table data, triggers, views, and stored procedures. The Include Create Schema option was enabled to ensure that the database can be fully recreated from the backup file if restoration is required. The export completed successfully, including all database objects.

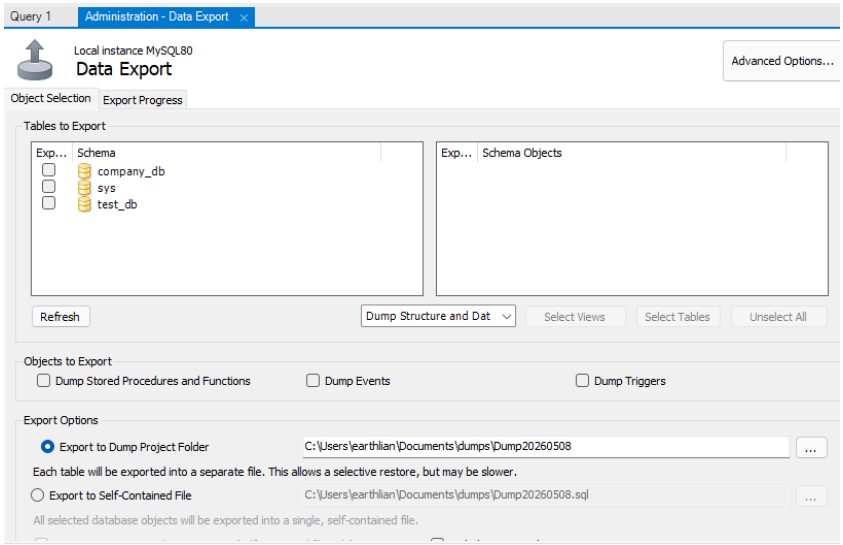


Figure 7.1: Preview of the export tab.

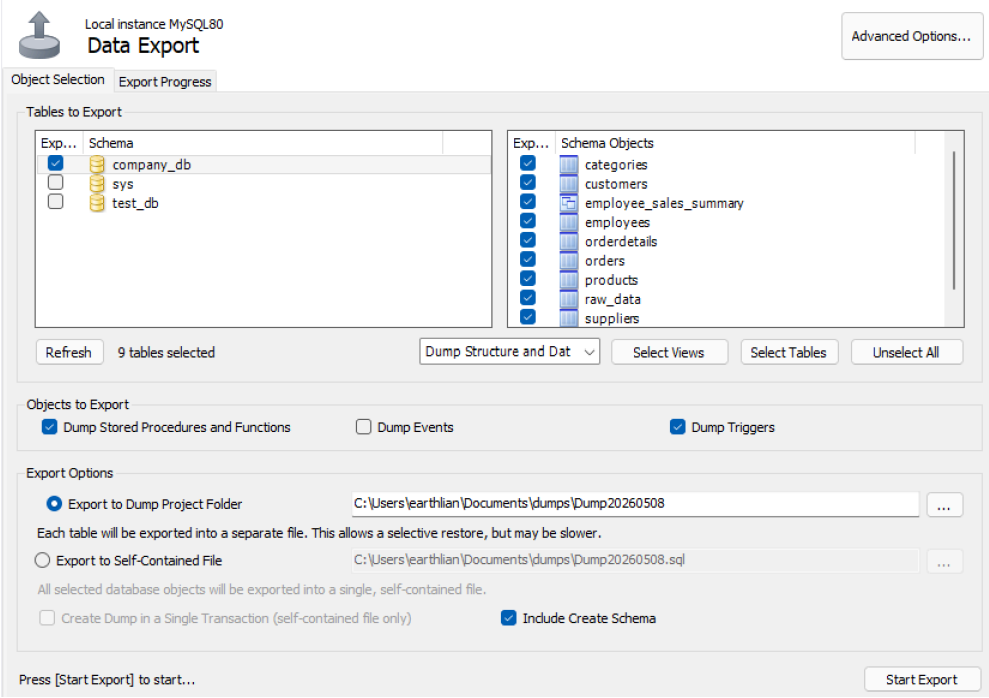


Figure 7.2: Selecting the company_db database to export and configuring the required settings.

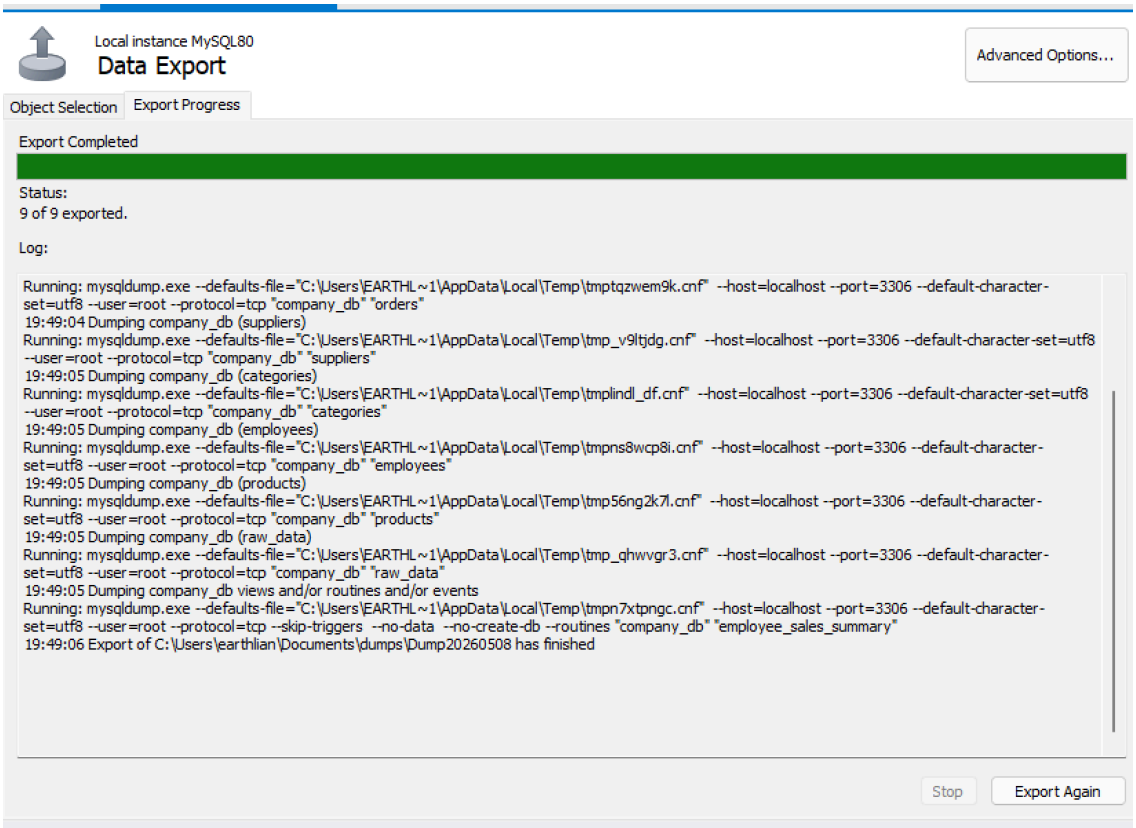


Figure 7.3: Output showing successful export.

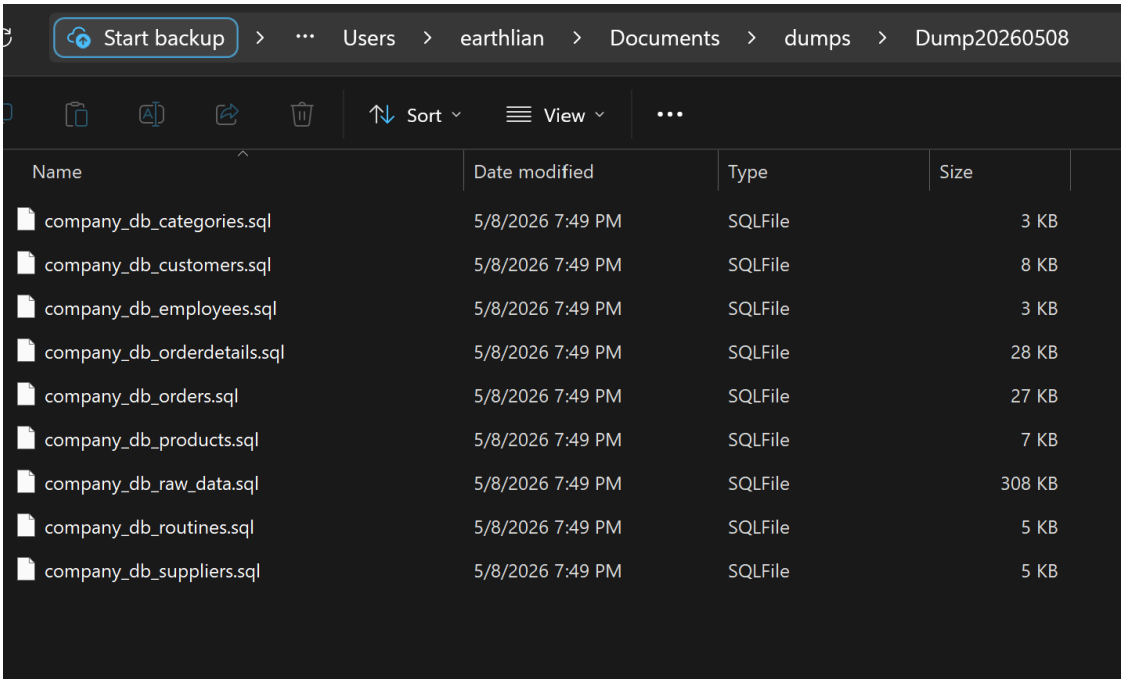


Figure 7.4: Screenshot showing backup on a local machine.

Group Contribution Table

Phase	Contributions
Database Creation & Data Loading	Jacob L. ● Cleaned up excel spreadsheet. ○ Changed column names. ○ Converted date formatting. ● Reviewed, proofread, and edited documentation. ● Code review and bugfixes to SQL script. Ipule P. ● Wrote documentation. ● Wrote SQL script to setup DB.
Database Diagram Design (ER)	Jacob L. ● Created initial ER diagram design. ● Provided ER rough draft figure. ● Reviewed, proofread, and edited documentation. Ipule P. ● Finalized ER diagram. ● Wrote documentation.
Normalization and Denormalization	Jacob L. ● Reviewed, proofread, and edited documentation. ● Code review and bugfixes to SQL script. ● Performed normalization testing. Ipule P. ● Implemented normalized table extraction SQL scripts. ● Documented normalization process and schema design decisions.
Table Alterations	Jacob L. ● Reviewed, proofread, and edited documentation. ● Code review and bugfixes to SQL script. ● Performed alterations testing. Ipule P. ● Modified table structures and data integrity constraints. ● Wrote documentation and exported table outputs.
Views, Triggers, and Stored Procedures	Jacob L. ● Reviewed, proofread, and edited documentation. ● Code review and bugfixes to SQL script. ● Commented trigger activation. ● Performed view, trigger, and store procedure testing. Ipule P. ● Designed and implemented SQL views, triggers, and stored procedures. ● Documented implementation process and outputs.

User Management and Privileges	Jacob L. ● Reviewed, proofread, and edited documentation. ● Code review and bugfixes to SQL script. ○ Edited analyst role. ● Ran tests for users and roles. Ipule P. ● Created database users and assigned privileges. ● Implemented role-based access permissions. ● Wrote documentation for privilege management.
Database Backup	Jacob L. ● Reviewed, proofread, and edited documentation. ● Performed database export and backup procedure tests. Ipule P. ● Performed database backup and export procedures. ● Wrote documentation for database backup
Other	Jacob L. ● Formatted, restructured, and proofread documentation. ● Drafted and finalized setup instructions in README.md. ● Created group contribution table. Ipule P. ● Wrote initial documentation. ● Created GitHub repository.

Link to Github repository: https://github.com/ipipi2024/business-db-project